연습문제 정답과 풀이

다중분류 (MNIST) · 딥러닝개론및연습 2026년 2학기

강의노트 끝의 연습문제 8개의 정답과 풀이이다. 문제를 다시 적고 그 아래에 풀이를 쓴다.

1. MNIST network에서 $l = 784$, $m = 100$, $n = 10$, 미니배치 크기 $h = 100$이다. X, $W^{(1)}$, $B^{(1)}$, A, Z, $W^{(2)}$, $B^{(2)}$, U, Y, T의 shape을 쓰고, 학습해야 하는 parameter의 총 개수를 구하라.

풀이. 미니배치의 데이터 하나가 한 행이므로 행의 수는 h이고, 열의 수는 그 층의 node 수이다.

변수	shape	변수	shape
X (입력)	$h \times l = 100 \times 784$	$W^{(2)}$	$m \times n = 100 \times 10$
$W^{(1)}$	$l \times m = 784 \times 100$	$B^{(2)}$	$1 \times n = 1 \times 10$
$B^{(1)}$	$1 \times m = 1 \times 100$	$U = ZW^{(2)} + B^{(2)}$	$h \times n = 100 \times 10$
$A = XW^{(1)} + B^{(1)}$	$h \times m = 100 \times 100$	$Y = \mathrm{softmax}(U)$	$h \times n = 100 \times 10$
$Z = h(A)$	$h \times m = 100 \times 100$	T (one-hot)	$h \times n = 100 \times 10$

$B^{(1)}$, $B^{(2)}$는 $1 \times m$, $1 \times n$ 행벡터이고, 더할 때 h개의 행에 broadcast된다.

학습하는 parameter는 $W^{(1)}, B^{(1)}, W^{(2)}, B^{(2)}$이다.

$$784 \cdot 100 + 100 + 100 \cdot 10 + 10 = 78400 + 100 + 1000 + 10 = \boxed{79510}.$$

(X, A, Z, U, Y, T는 데이터에 따라 정해지는 값이고 parameter가 아니다. 또 parameter의 개수는 미니배치 크기 h와 관계없다.)

2. $U = (1, 2, 3)$ 일 때 $Y = \mathrm{softmax}(U)$를 소수 셋째 자리까지 구하라. 또 임의의 상수 c에 대하여 $\mathrm{softmax}(U + c) = \mathrm{softmax}(U)$ 임을 보이라.

풀이. $e^1 \approx 2.718$, $e^2 \approx 7.389$, $e^3 \approx 20.086$이고 합은 30.193이다. 따라서

$$Y \approx \left(\frac{2.718}{30.193}, \frac{7.389}{30.193}, \frac{20.086}{30.193}\right) = \boxed{(0.090,\ 0.245,\ 0.665)}.$$

세 값의 합은 1이다. 점수가 1씩 커질 때마다 확률은 약 $e \approx 2.7$배씩 커진다.
$U + c = (u_1 + c, \ldots, u_n + c)$이면 $e^{u_i + c} = e^c e^{u_i}$이므로

$$\mathrm{softmax}(U + c)_i = \frac{e^c e^{u_i}}{\sum_j e^c e^{u_j}} = \frac{e^c e^{u_i}}{e^c \sum_j e^{u_j}} = \frac{e^{u_i}}{\sum_j e^{u_j}} = \mathrm{softmax}(U)_i.$$

분자와 분모의 e^c가 약분된다. 코드의 `U = U - np.max(U, axis=1, keepdims=True)`는 $c = -\max_j u_j$로 둔 것이다. 값은 그대로이면서 지수의 최댓값이 $e^0 = 1$이 되어 overflow가 생기지 않는다.

3. 정답이 $t = (0, 0, 1)$이고 network의 출력이 $Y = (0.2,\ 0.3,\ 0.5)$일 때 cross entropy 손실 l과 $\frac{\partial l}{\partial U}$를 구하라. 경사하강법은 세 점수 u_1, u_2, u_3를 각각 어느 방향으로 움직이는가?

풀이. t에서 1인 성분은 셋째뿐이므로

$$l = -\sum_i t_i \log y_i = -\log y_3 = -\log 0.5 = \log 2 \approx \boxed{0.693}.$$

softmax와 cross entropy를 묶은 계층의 역전파는 $\frac{\partial l}{\partial U} = Y - t$이므로

$$\frac{\partial l}{\partial U} = (0.2 - 0,\ 0.3 - 0,\ 0.5 - 1) = \boxed{(0.2,\ 0.3,\ -0.5)}.$$

경사하강법은 gradient의 반대 방향으로 움직인다 ($u_i \leftarrow u_i - \alpha \frac{\partial l}{\partial u_i}$).

- u_1, u_2: gradient가 양수이므로 **줄어든다**. 틀린 class의 점수를 낮춘다.
- u_3: gradient가 음수이므로 **커진다**. 정답 class의 점수를 높인다.

움직이는 양은 $y_i - t_i$에 비례한다. 즉 정답과 많이 다를수록 많이 움직인다. 또 세 성분의 합이 $0.2+0.3-0.5 = 0$이므로, 모든 점수에 같은 값을 더하는 방향으로는 움직이지 않는다 (문제 2와 같은 이유).

4. Affine 계층 $A = XW + B$에서

$$X = (1,\ 2), \qquad W = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \end{pmatrix}, \qquad B = (0,\ 1,\ 0), \qquad \frac{\partial l}{\partial A} = (1,\ -1,\ 2)$$

일 때 A, $\frac{\partial l}{\partial X}$, $\frac{\partial l}{\partial W}$, $\frac{\partial l}{\partial B}$를 구하라.

풀이. 순전파:

$$XW = (1{\cdot}1{+}2{\cdot}2,\ 1{\cdot}0{+}2{\cdot}1,\ 1{\cdot}(-1){+}2{\cdot}0) = (5,\ 2,\ -1), \qquad A = XW{+}B = \boxed{(5,\ 3,\ -1)}.$$

역전파 ($\frac{\partial l}{\partial X} = \frac{\partial l}{\partial A} W^\mathsf{T}$, $\frac{\partial l}{\partial W} = X^\mathsf{T} \frac{\partial l}{\partial A}$, $\frac{\partial l}{\partial B} = \frac{\partial l}{\partial A}$):

$$\frac{\partial l}{\partial X} = (1,\ -1,\ 2) \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ -1 & 0 \end{pmatrix} = (1 - 2,\ 2 - 1) = \boxed{(-1,\ 1)},$$

$$\frac{\partial l}{\partial W} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} (1,\ -1,\ 2) = \boxed{\begin{pmatrix} 1 & -1 & 2 \\ 2 & -2 & 4 \end{pmatrix}}, \qquad \frac{\partial l}{\partial B} = \boxed{(1,\ -1,\ 2)}.$$

shape 확인: $\frac{\partial l}{\partial X}$는 X와 같은 1×2, $\frac{\partial l}{\partial W}$는 W와 같은 2×3, $\frac{\partial l}{\partial B}$는 B와 같은 1×3이다.

5. softmax의 Jacobian $J = \left(\frac{\partial y_j}{\partial u_i}\right)$가

$$J = \mathrm{diag}(y_1, \ldots, y_n) - Y^\mathsf{T} Y$$

로 쓸 수 있음을 보이라. (Y는 $1 \times n$ 행벡터)

풀이. $S = \sum_k e^{u_k}$ 라 하면 $y_j = e^{u_j}/S$ 이고 $\frac{\partial S}{\partial u_i} = e^{u_i}$ 이다. 몫의 미분법으로

$$\frac{\partial y_j}{\partial u_i} = \frac{\delta_{ij} e^{u_j} S - e^{u_j} e^{u_i}}{S^2} = \delta_{ij} \frac{e^{u_j}}{S} - \frac{e^{u_j}}{S} \cdot \frac{e^{u_i}}{S} = \delta_{ij}\, y_j - y_i y_j.$$

(δ_{ij} 는 $i = j$ 이면 1, 아니면 0.) 첫째 항을 (i, j) 성분으로 갖는 행렬은 $\mathrm{diag}(y_1, \ldots, y_n)$ 이다. 둘째 항 $y_i y_j$ 는 열벡터 Y^T $(n \times 1)$와 행벡터 Y $(1 \times n)$의 곱 $Y^\mathsf{T} Y$ $(n \times n)$의 (i, j) 성분이다. 따라서

$$J = \mathrm{diag}(y_1, \ldots, y_n) - Y^\mathsf{T} Y.$$

J 는 대칭행렬이다. 또 각 행의 합이 $y_i - y_i \sum_j y_j = 0$ 인데, 이는 $\sum_j y_j = 1$ 이 항상 성립하기 때문이다.

6. ReLU 계층의 역전파를 다음과 같이 구현한 코드가 있다.

```
def backward(self, dZ):
    return np.maximum(self.A, 0) * dZ
```

무엇이 잘못되었는지 설명하고 올바르게 고쳐라.

풀이. ReLU $z = \max(0, a)$ 의 미분은 $a > 0$ 이면 1, $a \le 0$ 이면 0 이다. 따라서 역전파는 upstream gradient $\frac{\partial L}{\partial Z}$ 를 $a > 0$ 인 곳에서는 **그대로** 통과시키고 나머지에서는 0 으로 막아야 한다.
이 코드는 미분값 1 대신 **순전파의 출력값** $\max(0, a)$ 를 곱한다. 그래서 $a > 0$ 인 곳의 gradient 가 a 배가 된다. 예를 들어 $a = 3$ 이면 gradient 가 3 배로 부풀고, $a = 0.01$ 이면 $\frac{1}{100}$ 로 줄어든다. 연쇄법칙의 국소 미분을 함수값과 혼동한 것이다. (실습 코드의 `Relu` 계층은 순전파에서 `self.A`를 저장하지 않으므로, 이 코드는 그대로는 실행도 되지 않는다.)
순전파에서 저장한 `mask` $= (A > 0)$ 를 쓰면 된다.

```
def backward(self, dZ):
    return np.where(self.mask, dZ, 0.0)      # or: dZ * self.mask
```

7. 미니배치 학습에서 softmax–CE 계층이 `(Y - T) / h`를 돌려주는데, affine 계층의 `backward`에서도 `self.dW = self.X.T @ dA / h`처럼 다시 h 로 나누었다고 하자. $\frac{\partial L}{\partial W^{(2)}}$ 와 $\frac{\partial L}{\partial W^{(1)}}$ 는 정확한 값의 몇 배가 되는가?

풀이. 미니배치 손실 $L = \frac{1}{h}\sum_k l_k$ 의 $\frac{1}{h}$ 은 softmax–CE 계층의 `(Y - T) / h`에서 이미 한 번 들어갔다. 그 뒤의 계층은 연쇄법칙대로 곱하기만 하면 된다.

- 둘째 affine 계층: `dA`는 올바른 값인데 `dW`를 다시 h 로 나누므로 $\frac{\partial L}{\partial W^{(2)}}$ 는 정확한 값의 $\boxed{1/h}$ 배이다.
- 이 계층이 앞으로 돌려주는 `dA @ self.W.T`는 나누지 않았으므로 올바르다. ReLU 를 지나 첫째 affine 계층에 도착하는 `dA`도 올바르다. 그런데 첫째 affine 계층도 같은 클래스이므로 `dW`를 다시 h 로 나눈다. 따라서 $\frac{\partial L}{\partial W^{(1)}}$ 도 정확한 값의 $\boxed{1/h}$ 배이다.

$h = 100$ 이면 두 가중치의 gradient 가 모두 $\frac{1}{100}$ 이 된다. $\frac{\partial L}{\partial B}$ 는 나누지 않았으므로 올바르다. 결과적으로 가중치에 대해서만 학습률을 $\frac{1}{100}$ 로 줄인 것과 같아서 학습이 매우 느려진다. gradient 확인(gradient check)을 하면 수치 미분과 정확히 h 배 차이가 나서 바로 발견할 수 있다.

8. (도전) 손실함수를 제곱 손실 $l = \frac{1}{2}\sum_i (y_i - t_i)^2$ 으로 바꾸면

$$\frac{\partial l}{\partial u_i} = y_i\Big((y_i - t_i) - \sum_{j=1}^{n} y_j(y_j - t_j)\Big)$$

임을 보이라. 또 $R = Y - T$ 라 할 때 이 식을 미니배치 행렬로 쓰면

```
Y * (R - np.sum(Y * R, axis=1, keepdims=True))
```

가 됨을 확인하라.

풀이. l 은 $y_1, \ldots, y_n$ 을 거쳐 u_i 에 의존하므로 연쇄법칙으로

$$\frac{\partial l}{\partial u_i} = \sum_{j=1}^{n} \frac{\partial l}{\partial y_j}\frac{\partial y_j}{\partial u_i}, \qquad \frac{\partial l}{\partial y_j} = y_j - t_j.$$

문제 5에서 $\frac{\partial y_j}{\partial u_i} = \delta_{ij} y_j - y_i y_j$ 이므로

$$\begin{aligned}\frac{\partial l}{\partial u_i} &= \sum_j (y_j - t_j)(\delta_{ij} y_j - y_i y_j) = y_i(y_i - t_i) - y_i \sum_j y_j(y_j - t_j) \\ &= y_i\Big((y_i - t_i) - \sum_j y_j(y_j - t_j)\Big).\end{aligned}$$

미니배치 행렬. $Y, T, R = Y - T$ 는 모두 $h \times n$ 이고, k 번째 행이 k 번째 데이터이다.

- `Y * R`의 (k, j) 성분은 $y_{kj}(y_{kj} - t_{kj})$ 이다 (성분별 곱).
- `np.sum(Y * R, axis=1, keepdims=True)` 는 각 행의 합 $\sum_j y_{kj}(y_{kj} - t_{kj})$ 이고 shape은 $h \times 1$ 이다.
- `R - (...)`에서 $h \times 1$ 이 각 행의 n 개 성분에 broadcast되어 (k, i) 성분이 $(y_{ki} - t_{ki}) - \sum_j y_{kj}(y_{kj} - t_{kj})$ 가 된다.
- 마지막으로 `Y *`를 성분별로 곱하면 (k, i) 성분이 위에서 구한 $\frac{\partial l_k}{\partial u_{ki}}$ 이다.

`keepdims=True`가 없으면 shape이 $(h,)$ 가 되어 broadcast가 열 방향으로 일어나므로 틀린 값이 된다 ($h \neq n$ 이면 오류). 미니배치 평균 손실의 gradient는 여기에 $\frac{1}{h}$ 을 곱한 것이며, 실습 코드 `SoftmaxWithSquaredError.backward`는 마지막에 `/ h`를 한다.
cross entropy의 $Y - T$ 와 비교하면, 제곱 손실의 gradient에는 y_i 가 곱해져 있다. 그래서 정답 class의 확률 y_i 가 0에 가까운 (크게 틀린) 경우에 gradient가 작아져 학습이 느려진다.